

实验四：自定义内存分配器 (Malloc) 参考手册

本参考手册用于帮助你理解学生手册中的基础代码。请注意：基础实验只要求实现一个最小可用的 `malloc/free`，不要求实现块分割或块合并。这些内容会放在后面的进阶部分，作为二选一挑战的参考。

1. 基础部分：本实验必须掌握的知识

1.1 `malloc` 不是系统调用

在 Linux 中，内存分配可以粗略分为三个层次：

- **内核层**：负责管理物理内存、虚拟地址空间和页表。
- **系统调用层**：提供 `brk`、`sbrk`、`mmap` 等接口，让进程扩大或映射自己的地址空间。
- **标准库层**：实现 `malloc`、`free` 等函数，把系统调用拿到的大块内存管理成用户需要的小块内存。

本实验只做一个简化版标准库分配器：当没有可复用内存块时，通过 `sbrk` 扩大堆；当有空闲块可用时，直接复用它。

1.2 堆与 Program Break

堆（Heap）是进程地址空间中的一段区域，通常用于动态内存分配。堆顶端的位置叫 **Program Break**。

`sbrk` 用于移动或查询 Program Break：

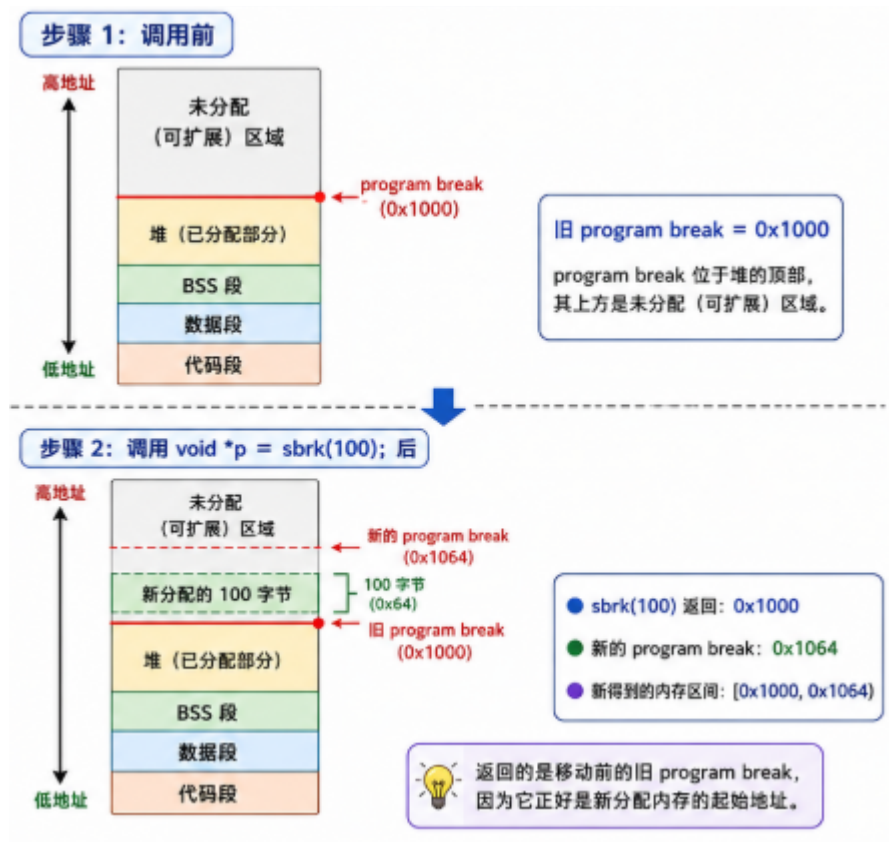
```
void *sbrk(intptr_t increment);
```

- `sbrk(0)`：不扩大堆，只返回当前 Program Break。
- `sbrk(n)`：将 Program Break 向后移动 `n` 字节，成功时返回移动前的旧 Program Break。
- 如果失败，返回 `(void *)-1`。

在本实验中，常见写法是：

```
block_meta *block = sbrk(0);
void *request = sbrk(META_SIZE + size);
if (request == (void *)-1) {
    return NULL;
}
```

这里的 `block` 是新内存块的起始地址。它先被用作元数据头部，紧跟在后面的区域才会返回给用户使用。



1.3 元数据：分配器自己藏起来的信息

用户调用 `my_malloc(20)` 时，只关心“我能不能拿到 20 字节”。但分配器还必须知道：

- 这个块有多大。
- 这个块现在是否空闲。
- 下一个块在哪里。

因此，每个用户数据区前面都放一个元数据结构：

```
typedef struct block_meta {
    size_t size;           // 用户数据区大小，不包含元数据头部
    int free;              // 1 表示空闲，0 表示正在使用
    struct block_meta *next; // 指向下一个块
} block_meta;
```

内存布局如下：

```
+-----+
| block_meta 元数据头部 | 用户真正可以读写的数据区 |
+-----+
^               ^
|               |
block          block + 1
```

`block` 指向元数据头部，`block + 1` 指向用户数据区起始位置。

1.4 申请空间时需要考虑元数据的原因

用户申请的 `size` 只表示用户数据区大小，但分配器还需要为 `block_meta` 预留空间。实现 `request_space` 时，要同时考虑：

元数据区域 + 用户数据区域

如果只为用户数据区申请空间，后续 `free` 时就无法从用户指针反推出块的大小和状态。

1.5 First-Fit：首次适配策略

本实验使用 **First-Fit（首次适配）** 策略寻找空闲块。

核心规则：

1. 从 `global_base` 指向的第一个块开始遍历链表。
2. 找到第一个 `free == 1` 且 `size` 足够大的块。
3. 如果找到了，就把它重新标记为已占用。
4. 如果没找到，就调用 `request_space` 向操作系统申请新堆空间。

示意代码：

```
while (current && !(current->free && current->size >= size)) {
    *last = current;
    current = current->next;
}
```

这里的 `last` 很重要。若最终没有找到空闲块，`current` 会变成 `NULL`，而 `last` 应该指向链表尾部。这样新申请的块才能通过 `last->next = block` 接回链表。

1.6 `my_malloc` 的基本流程

`my_malloc` 的核心流程如下：

```
用户请求 size
|
v
size == 0 ? 返回 NULL
|
v
global_base 是否为空？
|
+-- 是：第一次分配，调用 request_space，并设置 global_base
|
+-- 否：调用 find_free_block
      |
      +-- 找到空闲块：free 改为 0，复用旧块
      |
```

+++ 没找到：调用 `request_space`，在链表尾部追加新块

```
|  
v  
返回 block + 1
```

最后返回 `block + 1`，是为了跳过元数据头部，只把用户数据区暴露给调用者。

1.7 `my_free` 的基本流程

`my_free(ptr)` 收到的是用户数据区地址，而不是元数据地址。因此需要做一次反向指针运算：

```
block_meta *block = (block_meta *)ptr - 1;  
block->free = 1;
```

本实验中的 `free` 只做一件事：把对应块标记为空闲。

它不会：

- 清空用户数据区内容。
- 把堆空间立刻还给操作系统。
- 合并相邻空闲块。

因此，释放后再次分配到同一块内存时，可能看到旧数据残留。这是本实验需要观察的现象之一。

2. 基础实验的局限性

本实验的基础代码故意保持简单，因此它有明显局限：

- 复用大空闲块满足小请求时，会浪费块内剩余空间。
- 释放相邻块后，它们仍然是链表中的两个独立块。
- 没有检查重复释放、野指针释放等错误。
- 没有处理多线程并发访问。
- 没有做严格的内存对齐。

这些不是基础代码的错误，而是为了让你先掌握最小分配器的主干逻辑。真实的 `malloc` 会围绕这些问题继续扩展。

3. LD_PRELOAD 动态拦截知识

`LD_PRELOAD` 是 Linux 动态链接器提供了一种预加载机制。程序启动时，动态链接器会先把 `LD_PRELOAD` 指定的共享库加载进进程地址空间，再加载程序本身依赖的其他共享库。这样一来，如果预加载库中定义了和 `libc` 同名的函数，例如 `malloc`、`free`，动态链接器在解析符号时会优先命中预加载库中的版本。

本实验利用这一点，把系统命令中的 `malloc/free/calloc/realloc` 调用转发到自己实现的分配逻辑，从而观察“函数调用被替换”的效果。

3.1 动态链接与符号优先级

普通 C 程序在运行时会依赖动态链接器加载共享库，例如 `libc`。程序里调用 `malloc` 时，最终需要解析到某个共享库中的 `malloc` 符号。

使用：

```
LD_PRELOAD=./hook_malloc.so ls -al
```

时，`hook_malloc.so` 会被提前加载。只要其中导出了名为 `malloc`、`free`、`calloc`、`realloc` 的函数，`ls` 在运行过程中触发的动态内存分配就会优先进入实验代码中的拦截函数。

需要注意，`LD_PRELOAD` 主要影响动态链接程序。静态链接程序不会按这种方式解析共享库符号；某些带有安全限制的程序也可能忽略或限制 `LD_PRELOAD`。

3.2 拦截函数的基本要求

拦截函数必须和原函数保持兼容的函数名与函数签名：

```
void *malloc(size_t size);
void free(void *ptr);
void *calloc(size_t nmemb, size_t size);
void *realloc(void *ptr, size_t size);
```

函数名用于覆盖 `libc` 中的同名符号，参数和返回值用于保证调用约定一致。如果签名不匹配，调用方仍会按照原始 `malloc/free` 的方式传参和取返回值，容易导致未定义行为。

共享库通常使用下面的方式编译：

```
gcc -shared -fPIC hook_malloc.c my_malloc.c -o hook_malloc.so -pthread
```

- `-shared`：生成共享库，而不是普通可执行文件。
- `-fPIC`：生成位置无关代码，便于共享库被加载到不同地址。
- `hook_malloc.c`：提供同名的 `malloc/free/calloc/realloc` 拦截入口。
- `my_malloc.c`：提供真正执行分配和释放的实现。

3.3 拦截后的调用路径

以 `malloc` 为例，使用 `LD_PRELOAD` 后的调用路径可以理解为：

```
ls 内部调用 malloc
|
v
动态链接器优先解析到 hook_malloc.so 里的 malloc
|
v
拦截函数输出提示信息
|
v
调用 my_malloc 完成实际分配
|
v
把 my_malloc 返回的用户指针交还给 ls
```

`free` 的路径类似：程序调用 `free`，先进入 `hook_malloc.so` 中的 `free`，再由拦截函数调用 `my_free` 标记内存块为空闲。

本实验中的 `calloc` 和 `realloc` 不是继续调用 `libc` 的原始实现，而是建立在 `my_malloc/my_free` 之上：

- `calloc(nmemb, size)`：先检查 `nmemb * size` 是否溢出，再申请总大小为 `nmemb * size` 的内存，最后把用户数据区清零。
- `realloc(NULL, size)`：等价于调用 `hook` 中自己定义的 `malloc(size)`，最终进入 `my_malloc`。
- `realloc(ptr, 0)`：等价于调用 `hook` 中自己定义的 `free(ptr)`，最终进入 `my_free`，然后返回 `NULL`。
- `realloc(ptr, size)`：如果旧块足够大，可以直接返回原指针；如果旧块不够大，需要通过自己的 `malloc/free` 申请新块、复制旧数据、释放旧块。

3.4 运行输出的含义

使用新的 `hook_malloc.c` 运行：

```
LD_PRELOAD=./hook_malloc.so ls -al
```

屏幕上可能出现大量类似下面的提示：

```
Haha! I intercepted malloc!
Haha! I intercepted free!
Haha! I intercepted realloc!
Haha! I intercepted calloc!
```

这些提示通常会和 `ls` 原本的目录列表交错出现。原因是拦截函数使用 `write(STDOUT_FILENO, ...)` 直接向标准输出写入提示信息，而 `ls` 本身也在向标准输出打印目录内容。两类输出写到同一个文件描述符后，就会按实际执行顺序混在一起。

大量 `malloc/free` 提示说明 `ls` 这样的普通命令在启动、读取目录、格式化输出、清理资源时都会频繁使用动态内存。`realloc` 提示说明程序或运行时库会扩展已有缓冲区。`calloc` 提示说明有些路径需要申请并清零一段内存，例如初始化表项或内部状态结构。

目录内容打印完成后仍然可能继续出现 `free` 提示。这通常来自程序退出阶段的资源清理，属于正常现象。

4. 进阶部分：二选一挑战参考

进阶挑战只要求从下面两个方向中选择一个完成。参考手册不会直接给出完整代码，而是说明需要用到的知识和实现提示。

4.1 挑战一：块的分割（Splitting）

相关知识

基础实现中，`find_free_block` 只要找到一个足够大的空闲块，就会把整块都交给这次 `my_malloc` 使用。

例如，一个 1000 字节的空闲块被 `my_malloc(10)` 复用后，基础实现会把整个 1000 字节都标记为正在使用。用户实际只需要 10 字节，剩下的大部分空间无法再分配给其他请求，这属于 **内部碎片**。

块分割的思路是：当一个空闲块明显大于本次申请大小时，只取前半部分给用户，把剩余空间重新组织成一个新的空闲块，并插回链表。

```
分割前：
[ free block: 1000B ]

my_malloc(10)

分割后：
[ used block: 10B ][ free block: 剩余空间 ]
```

需要注意：新空闲块也需要自己的 `block_meta`。因此，剩余空间必须至少能放下一个 `block_meta`，并且还要有一点可用数据区，否则不值得分割。

实现提示

1. 分割逻辑适合放在 `my_malloc` 中“找到可复用空闲块”之后、`block->free = 0` 之前或附近。
2. 可以先判断当前空闲块是否足够大，例如：

```
block->size >= size + META_SIZE + 1
```

3. 新空闲块的位置可以通过指针偏移得到。思路是从当前块的数据区开始，跳过本次申请的 `size` 字节，那里就是新元数据的位置。
4. 新空闲块的 `size` 应该是原块剩余空间扣掉 `META_SIZE` 后的大小。
5. 分割后要维护链表：
 - 新空闲块的 `next` 指向原来 `block->next`。
 - 当前块的 `next` 指向新空闲块。
 - 当前块的 `size` 改成本次申请的 `size`。
6. 最后再把当前块标记为正在使用。

可以重点测试：释放一个较大的块后，再申请一个较小的块，观察剩余部分是否还能被下一次申请继续复用。

4.2 挑战二：块的合并 (Coalescing)

相关知识

基础实现中，`my_free` 只做一件事：把当前块的 `free` 标记改为 1。即使两个相邻的块都已经空闲，它们在链表中仍然是两个独立的小块。

例如，连续两个 100 字节块都被释放后，如果程序申请 180 字节，基础实现仍然可能找不到可用块，因为链表里没有单个大小达到 180 字节的空闲块。这属于 **外部碎片**。

块合并的思路是：释放一个块后，如果它后面的相邻块也是空闲的，就把两个块合并成一个更大的空闲块。

```
合并前：
[ free block: 100B ][ free block: 100B ]

合并后：
[ free block: 100B + META_SIZE + 100B ]
```

合并后的大小需要加上 `META_SIZE`，因为第二个块原来的元数据区域也变成了大空闲块的一部分。

实现提示

1. 合并逻辑适合放在 `my_free` 中，将当前块标记为空闲之后。
2. 在单向链表中，最容易先实现“向后合并”：检查 `block->next` 是否存在且也是空闲块。
3. 如果 `block->next` 是空闲块，可以执行类似这样的更新思路：

```
block->size = block->size + META_SIZE + block->next->size;
block->next = block->next->next;
```

4. 如果想继续合并多个连续空闲块，可以使用 `while` 循环反复检查 `block->next`。
5. “向前合并”需要找到当前块的前一个节点。由于基础链表是单向链表，不能直接从当前块往前走，需要从 `global_base` 开始遍历找前驱节点。
6. 二选一挑战中，可以只实现向后合并；如果同时实现向前合并，需要特别注意不要把链表接断。

可以重点测试：连续释放两个相邻的小块后，再申请一个比单个小块更大、但小于合并后大小的块，观察是否能成功复用合并后的空闲块。